

BusGo (Foundation) - Capstone Project for Certification

Contents

Purpose

1) About Us

2) Problem Statement

3) Roles

4) In-Scope Features

5) User Flows (Happy Path)

7) Architecture

8) Tech Stack

9) Data Model

10) API Outline

11) NFRs & SLAs

12) Testing

13) Engineering PoV

14) Delivery Plan

15) Sample Acceptance Criteria

16) Example Seed Data (SQL)

17) Simple Wireframe

18) Sample User Stories

19) Definition of Ready (DoR) & Done (DoD)

20) Evaluation Rubric (Aligned to Foundation Skills)

22) Stretch Goals (Optional, If Time Permits)

23) Getting Started (Suggested Commands)

Purpose

Build a bus ticket booking web application where users can search for available buses between two cities, choose a departure, pick seats, and confirm a booking. Admins manage bus routes and schedules. The focus is on foundational full-stack skills - React/Angular, Spring Boot, REST, PostgreSQL, Git, and basic testing - aligned with the Foundation skill matrix.

1) About Us

At BusGo, we make intercity travel simple. This Foundation version delivers a clean route-search, seat-selection, and booking experience - no payment gateway, no real-time GPS tracking, just a complete booking flow.

2) Problem Statement

Travellers need a simple way to search buses between cities, view available seats, and confirm a ticket - without visiting a booking counter.

Goal: Deliver a minimal yet complete bus booking flow demonstrating full-stack fundamentals.

3) Roles

1. Passenger/User - Can search routes, view seat layout, book and cancel tickets.
2. Admin - Can manage buses, routes, and schedules.

Note: Simple isAdmin boolean flag on the User entity.

Role Flag / Field Approach

- User entity: boolean isAdmin; (defaults to false on registration)
- Seed one admin user with isAdmin = true.
- Backend: check isAdmin before allowing bus/route/schedule CRUD.
- Frontend: show Admin menu only when isAdmin is true in user context.

Backend Concept

- Backend: check isAdmin before allowing bus/route/schedule CRUD.
- Frontend: show Admin menu only when isAdmin is true in user context.

Frontend Concept

- Backend: check isAdmin before allowing bus/route/schedule CRUD.
 - Frontend: show Admin menu only when isAdmin is true in user context.
-

4) In-Scope Features

Authentication (Basic)

- Register, Login, Logout.
- Simple password hashing (BCrypt).

Route Search

- Search form: From City, To City, Date.
- Results list: bus name, operator, departure time, arrival time, seats available, fare.
- Click a result to view seat layout.

Seat Selection

- Bus layout: 10 rows x 4 seats (40 total) - simple grid.
- Seats shown as Available or Booked.
- User selects 1-4 seats and proceeds to confirm.

Booking & My Trips

- Confirm booking -> seats marked BOOKED.
- My Trips page: list of bookings with route, date, seat numbers, status.
- Cancel a booking (seats freed, status = CANCELLED).

Admin - Bus & Schedule Management

- CRUD for buses (busNumber, operatorName, totalSeats, busType: SLEEPER/SEATER).
 - CRUD for schedules (busId, fromCity, toCity, departureTime, arrivalTime, fare, journeyDate).
-

5) User Flows (Happy Path)

1. Search: User enters From, To, Date -> sees list of available buses.
 2. Select: Clicks a bus -> sees seat layout with availability.
 3. Auth: If not logged in -> prompted to register/login.
 4. Confirm: Selects seats -> reviews fare total -> confirms booking.
 5. My Trips: Views all bookings -> can cancel upcoming trips.
 6. Admin: Logs in -> adds buses and schedules.
-

7) Architecture

- Frontend: React (Hooks + Context API) or Angular (Services + Components). Routing enabled.
 - Backend: Spring Boot (REST Controllers, Services, Spring Data JPA).
 - Database: PostgreSQL (Dev may use H2 for quick start).
 - Auth: Stateless JWT or simple session cookie (choose one; document trade-offs).
 - State: Selected seats held in frontend state; no seat-lock TTL at Foundation level.
 - Deployment: Localhost
-

8) Tech Stack

Frontend

- React + Vite/CRA (Routing, Hooks, Context) or Angular (Routing, Services)
- HTML5, CSS3, JavaScript (ES6), TypeScript

Backend

- Java 17+, Spring Boot, Spring Web, Spring Data JPA, Validation

- JUnit5, Mockito

Database

- PostgreSQL, JPA/Hibernate
- H2 for local

Tools & Practices

- Git Basics (branching, PRs), Postman/Swagger, Agile ceremonies
 - AI Pair Programming: GitHub Copilot/ChatGPT for boilerplate and explanations
-

9) Data Model

Entities & Relationships

- Bus (1) -- (M) Schedule
- User (1) -- (M) Booking
- Schedule (1) -- (M) Booking
- Booking (1) -- (M) BookingSeat (seatNumber stored as String e.g. '2A', '3B')

Note: No separate Seat table. Booked seats derived from BookingSeat rows per schedule.

10) API Outline

Auth

- POST /api/auth/register - {email, password, name} -> 201
- POST /api/auth/login -> 200 {token}
- POST /api/auth/logout -> 204

Routes/Schedules

- GET /api/schedules?from=&to;=&date;= - search available schedules
- GET /api/schedules/{id}/seats - booked seat numbers for a schedule
- POST /api/schedules (admin) - create
- PUT /api/schedules/{id} (admin)
- DELETE /api/schedules/{id} (admin)

Buses

- GET /api/buses (admin)
- POST /api/buses (admin)
- PUT /api/buses/{id} (admin)
- DELETE /api/buses/{id} (admin)

Bookings

- POST /api/bookings - {scheduleId, seatNumbers[]} -> creates booking
- GET /api/bookings/mine - user's bookings
- PUT /api/bookings/{id}/cancel - cancel booking

Error Format (Consistent)

```
{
  "timestamp": "2025-09-12T12:00:00Z",
  "path": "/api/...",
  "error": "VALIDATION_ERROR",
  "message": "Field is required"
}
```

11) NFRs & SLAs

- Performance:
 - API response ≤ 2 s under light load (1-5 concurrent users locally).
 - UI renders initial content within 3 s on localhost.
- Security:
 - Passwords hashed (BCrypt); JWT/session token stored securely.
 - Basic input validation & server-side checks.
- Scalability/Concurrency:
 - Test with 5-10 concurrent requests locally. Simulate using Postman Runner or multiple browser tabs. No JMeter or load-testing tools required at this level.
- Error Handling:
 - User-friendly error toasts/messages on UI; structured error JSON on API.
- Logging:
 - Console logs; simple request/response logs (no external logging infra).
- Documentation:
 - README.md with setup steps, seed data, and run commands.
 - API contracts documented with Swagger/OpenAPI.

12) Testing

- Backend Unit Tests
 - Services & Repos with JUnit5 + Mockito
 - Target coverage: $\geq 60\%$ (method-level)
- API Testing
 - Manual via Postman or Swagger.
- Frontend
 - Optional unit tests for 1-2 key components (e.g., Booking form).
- No automated regression/e2e required.

13) Engineering PoV

- Code Quality:
 - Follows SOLID basics where practical (e.g., service layer, DTOs).
 - No critical lint errors (ESLint/TSLint).
- Branching:
 - main (stable), feature/* branches; PR reviews by peers/mentor.
- Coverage:
 - Aim 60-70% backend, frontend minimal.
- Dependencies:

- Keep list short; beware of transitive vulnerabilities.
 - Docs:
 - API endpoints documented; Postman collection included.
-

14) Delivery Plan (3 Sprints, 1-2 weeks each)

Sprint 1 - Foundations & Route Search

- Scaffolding (FE + BE), DB setup, Git workflow
- Bus entity, Schedule entity; GET /schedules?from=&to;=&date;=
- Frontend: Home + Search Form + Results List
- Seed data loading (cities, buses, schedules)

Sprint 2 - Auth & Seat Selection

- Register/Login; protect booking routes
- Seat layout UI; GET booked seats; seat selection logic
- Fare calculation (seats x fare); validation + error handling

Sprint 3 - Booking & Admin

- POST /api/bookings; My Trips page; Cancel booking
 - Admin: Bus CRUD + Schedule CRUD screens
 - Polish: README, Postman collection, unit tests >=60%
-

15) Sample Acceptance Criteria

Route Search

- Given I search Hyderabad -> Bangalore on a seeded date, I see at least 2 buses.
- Results show operator name, departure/arrival time, and fare.

Seat Selection

- Given I open a schedule, I see a 10x4 seat grid with booked seats highlighted.
- I cannot select an already-booked seat.

Booking

- After confirming, my seat numbers appear as booked for that schedule.
- Cancelling a booking changes status to CANCELLED and frees the seats.

Admin

- Only admin can access /admin.
 - Admin adds a schedule and it appears in search results for the correct date.
-

16) Example Seed Data (SQL)

SQL

```
INSERT INTO users(id,email,password_hash,is_admin,name,created_at)
VALUES (gen_random_uuid(),'admin@busgo.com','$2a$10$hash',true,'Admin',now());

INSERT INTO bus(id,bus_number,operator_name,total_seats,bus_type,created_at)
```

```
VALUES
(gen_random_uuid(), 'KA-01-AB-1234', 'VRL Travels', 40, 'SEATER', now()),
(gen_random_uuid(), 'TN-02-CD-5678', 'SRS Travels', 40, 'SLEEPER', now());

INSERT INTO schedule(id, bus_id, from_city, to_city, departure_time, arrival_time, fare, journey_date, created_at)
VALUES
(gen_random_uuid(), (SELECT id FROM bus WHERE bus_number='KA-01-AB-1234'),
'Hyderabad', 'Bangalore', '2025-10-20 21:00:00', '2025-10-21 06:00:00', 650.00, '2025-10-20', now()),
(gen_random_uuid(), (SELECT id FROM bus WHERE bus_number='TN-02-CD-5678'),
'Hyderabad', 'Bangalore', '2025-10-20 22:30:00', '2025-10-21 07:30:00', 750.00, '2025-10-20', now());
```

17) Simple Wireframe

- Home: Search panel (From City / To City / Date / [Search Buses])
- Search Results: Cards showing Bus Name - Operator - Departure - Arrival - Fare - Seats Left - [Select Seats]
- Seat Layout: 10 rows x 4 columns grid. Available = green, Booked = red. Fare summary + [Confirm]
- My Trips: Table (Route / Date / Seats / Fare / Status) - [Cancel Trip]
- Admin: Buses table + Schedules table + Add/Edit forms

18) Sample User Stories

- As a passenger, I want to search buses by origin, destination, and date so I can plan my journey.
- As a passenger, I want to see available seats so I can choose my preferred seat.
- As a passenger, I want to confirm a booking so I can secure my ticket.
- As a passenger, I want to cancel a trip so I can free my seat if plans change.
- As an admin, I want to add a new schedule so passengers can book the new route.

19) Definition of Ready (DoR) & Done (DoD)

DoR

- User story has clear acceptance criteria.
- API/DB changes identified (if any).
- Mock/UX reference exists (even a sketch).

DoD

- Code compiles; unit tests for backend services pass.
- Manual tests completed (Postman + UI).
- No critical ESLint/TS issues; meaningful README update.
- Peer review completed; merged to main.

20) Evaluation Rubric (Aligned to Foundation Skills)

Area	What We Check	Weight
Frontend (React/Angular)	Routing, hooks/services, component communication, state handling (Context/Service), basic TypeScript usage	25%

Area	What We Check	Weight
Backend (Spring Boot)	Clean REST controllers, services, JPA entities & relationships, validation, basic error handling	25%
Common Skills	Core Java correctness, SQL/JPA usage, SOLID basics, Git hygiene, HTML/CSS semantics	20%
Testing	JUnit/Mockito for services/repos; Postman collection; target $\geq 60\%$ coverage	15%
Generative AI Usage	Appropriate use of Copilot/ChatGPT for boilerplate, explanations, refactoring (cite prompts in README)	10%
Documentation & Readme	Setup steps, seed data, API list, constraints & trade-offs	5%

22) Stretch Goals (Optional, If Time Permits)

- City autocomplete dropdown on the search form.
- Sort results by fare or departure time.
- Booking summary page with a printable ticket layout.

23) Getting Started (Suggested Commands)

Backend

- Java 17+, Spring Boot.
- Profiles: dev (H2), prod (Postgres).

Frontend

- React: `npm create vite@latest -ui -- --template react-ts`
- Angular: `ng new -ui --routing --style=scss`

Run Order

1. Start DB (H2 or Postgres) 2. Run Spring Boot 3. Seed Data 4. Run UI